Stakeholder Agent System

Quality Assurance Report

Regis University - MSSE 692: Software Engineering Practicum I

Matt Bass, Anthony Perez, Joseph Turner, Ramesh Kondala3-1-2026

Testing Strategy & Results

Testing approach

The Stakeholder Agent System employs a layered testing strategy focused primarily on unit and component-level validation, supplemented by service-level integration testing and structured manual validation. The strategy prioritizes correctness of business logic, authentication security, and AI orchestration reliability while isolating external dependencies such as LLM inference.

Testing was divided between backend (FastAPI AI service) and frontend (TanStack-based UI).

Backend Testing

Backend tests were implemented using pytest with coverage measurement via pytest-cov

The backend testing strategy primarily consists of:

Unit Tests

Unit tests validate individual services and logic components in isolation. These include:

- Persona loading
- Project loading
- History summarization logic
- Agent orchestration logic
- LLM error handling
- Message persistence and retrieval
- JWT validation and authentication edge cases
- User dependency resolution

All external LLM calls were mocked, ensuring:

- Deterministic test behavior
- No dependency on third-party API availability
- Isolation of internal logic from inference latency

Service-Level Integration Tests

Service-level tests validate realistic agent execution flows including:

- Empty conversation history scenarios
- Persona preservation across turns
- Error propagation paths

These tests simulate end-to-end service logic within the application layer but do not execute full HTTP request cycles.

What Was Not Tested

The following were not implemented during this sprint:

- Full HTTP-level integration tests using FastAPI's TestClient
- Automated load testing

- Concurrency testing
- Performance stress testing beyond log measurement

These were deferred due to sprint scope constraints and MVP prioritization.

Frontend Testing

Frontend tests were implemented using Vitest with TypeScript support.

Testing focuses on:

Component Tests

- RootErrorBoundary behavior
- RouteErrorBoundary behavior
- Error fallback components
- Error overlay rendering
- Conversation schema validation
- Custom hook behavior (useErrorLogger)

Hook-Level Tests

Environment-aware logging behavior was validated:

- Development vs production session storage
- Structured error logging

Integration Tests

Frontend integration tests validate that:

- Error boundaries wrap routes correctly
- Proper contextual error information renders during failures

What Was Not Tested

- No browser-level end-to-end tests (e.g., Cypress, Playwright) were implemented
- No automated accessibility testing tools were integrated
- No performance profiling of frontend rendering

Manual Testing

Manual exploratory testing was performed throughout development. Developers executed:

- Full conversation creation and resume workflows
- Authentication flows
- Message submission and AI response validation
- Error boundary triggering scenarios

This helped validate system behavior under real user conditions, especially around AI latency and UI responsiveness.

Testing in the CI Pipeline

The backend uses:

- Pytest for automated testing
- Pytest-cov for coverage reporting
- Mypy for static type checking
- Ruff for linting and formatting
- Pre-commit hooks enforcing code standards

Frontend CI includes:

- Vitest execution
- TypeScript static type checking
- Validation of zero TypeScript compilation errors

All tests must pass before code integration.

Test coverage metrics

Backend Coverage

Backend tests now achieve:

- **Total backend coverage: 100%**
- Core service layer coverage: 100%
- Authentication & schema validation: 100%
- History compactor service: 100%
- Agent service: 100%
- Repository abstractions: 100%
- Middleware event tracing: 100%
- Security internals: 100%
- Application entry point (main.py): 100%

This represents full statement and branch coverage across all backend modules, including previously under-tested infrastructure components such as middleware, security utilities, and application bootstrapping logic.

Achieving 100% coverage required:

- Additional tests for repository base abstractions
- Middleware execution and trace event validation tests
- Security utility validation paths
- Explicit tests for application startup behavior
- Expanded negative-path and edge-case testing

With full backend coverage, all service logic, infrastructure wiring, and error-handling paths are now verified through automated testing.

Frontend Coverage

Frontend testing results:

- Total test files: 20

- Total tests: 96
- All tests passed
- No TypeScript errors detected

Frontend coverage percentage was not generated during this run

Total System Coverage

With backend coverage increased to 100%, overall system coverage has significantly improved.

- Backend coverage: 100%
- Frontend coverage: 78%
- Total combined coverage (weighted by codebase size): ~89%

While backend coverage is now comprehensive, frontend coverage remains comparatively low. This is primarily due to:

- Many presentational UI components not yet included in coverage exclusions
- Limited automated testing of purely visual components
- Absence of browser-level end-to-end testing

The backend now has full automated validation across business logic, infrastructure wiring, authentication, and application entry points. The primary opportunity for increasing overall system coverage lies in expanding frontend component and integration tests.

Tools Used

Backend:

- pytest
- pytest-cov
- mypy
- Ruff

Pre-commit hooks

Frontend:

- Vitest
- TypeScript compiler

Test Results

Backend

- Total tests: 96
- Passed: 96
- Failed: 0
- Pass rate: 100%

Frontend

- Total tests: 35
- Passed: 35
- Failed: 0
- Pass rate: 100%

Overall System

- Total tests across system: 131
- Total passed: 131
- Total failed: 0
- Overall pass rate: 100%

The increase in backend testing raised total system tests from 79 to 120, while maintaining a 100% pass rate across both frontend and backend components.

Example Critical Test Cases

1. JWT Authentication Validation
    - Verifies rejection of invalid Bearer tokens
    - Confirms user ID extraction from valid tokens
2. Agent Orchestration Logic
    - Ensures persona instructions persist across turns
    - Validates correct summarization when history exceeds threshold
3. LLM Error Handling
    - Simulated API failures confirm graceful exception handling
4. History Summarization
    - Confirms that when message count > 10, history compaction occurs
    - Ensures summarization output is injected into prompt
5. Error Boundary Rendering (Frontend)
    - Simulates thrown errors to validate fallback UI behavior

Integration testing

Consolidated with Front End testing in Biome. Those test counts include the Integration Tests.

End-to-End

End to End testing is set up to run playwright on the staging environment. We test all critical paths for the application.

Search tests

All 5 | Passed 5 | Failed 0 | Flaky 0 | Skipped 0

2/28/2026, 4:48:33 PM Total time: 23.3s

auth.setup.ts

authenticate setup auth.setup.ts:6		1.8s

chat.spec.ts

chat › home page loads and shows new conversation button chromium chat.spec.ts:4		2.2s
chat › can create a conversation chromium chat.spec.ts:9		2.3s
chat › can send a message and receive a response chromium chat.spec.ts:22		18.2s
chat › messages persist on page reload chromium chat.spec.ts:46		17.5s

User Acceptance Testing

Acceptance testing was conducted as a requirements-driven validation exercise to confirm that the system satisfied the functional objectives defined for the Minimum Viable Product (MVP). The team used the consolidated project requirements document as the authoritative reference for determining completeness.

Each team member independently reviewed the requirements list and systematically validated the presence of each specified capability within the deployed application. For every requirement, the tester executed representative user workflows designed to exercise the described functionality, observed system behavior, and confirmed whether the outcome aligned with the stated expectation. Requirements that were successfully demonstrated were marked as complete; Fortunately, all the required functionality was confirmed, and no deficiencies were discovered through the acceptance testing process.

This distributed validation approach provided informal cross-verification. Differences in interpretation or observed behavior were reconciled during team discussion, and any issues identified were entered into the bug register and categorized according to the established severity scale. The process focused on confirming functional completeness rather than performing exhaustive edge-case or load testing.

At the conclusion of acceptance testing, all intended MVP functionality was confirmed as implemented and operational. During this review, a small number of requirements were identified that had not been properly scoped out of the original specification and therefore exceeded the defined MVP boundary. These items were reclassified as post-MVP enhancements rather than defects.

Overall, the acceptance testing process provided structured confirmation that the delivered system meets its defined MVP scope, with minor refinements and future enhancements clearly documented for subsequent development cycles.

Bug Report & Resolution

Front End Bugs

(P1) Rate Limiting missing (remedied)

The rate limiting mechanism to regulate requests to the front-end API has not been implemented.

(P3) Optimistic UI Message Persistence on API Failure

The front-end chat interface employs an optimistic update strategy. When a user submits a message to the AI service, the message is immediately rendered in the chat history before confirmation of successful API response. While this improves perceived responsiveness under normal operating conditions, a failure scenario exposes a consistency issue: if the API call fails, the message remains in the chat history without confirmation that it was processed by the backend.

From a UX and data integrity perspective, this creates ambiguity. The user cannot reliably determine whether the message was received, processed, or discarded. The appropriate corrective behavior would involve either (a) rolling back the optimistic update upon failure, or (b) clearly marking the message as failed and prompting retry.

(P3) Safari Input Highlighting Obscures Login Credentials

Manual browser testing revealed a Safari-specific rendering issue in which text highlighting behavior in username and password fields obscures entered data. This appears to be related to default focus and highlight styles interacting with custom CSS.

Although this does not prevent login functionality, it degrades usability and may reduce user confidence in the authentication flow. Because the defect is browser-specific and does not affect data correctness or system stability, it was classified as P3. Remediation likely requires refinement of CSS focus states and cross-browser validation.

(P4) Create Conversation Button Text Responsiveness

At reduced viewport widths, the “Create Conversation” button does not dynamically adjust its font size. As a result, the button label becomes partially truncated on smaller screens.

This defect is strictly cosmetic and does not affect functional behavior. However, it detracts from responsive design quality and overall polish. It was classified as P4 (low priority, minor visual defect).

Back End Bugs

(P2) Insufficient Observability in Wide Events

The current events instrumentation lacks contextual depth. “Wide-events” capture high-level service timing metrics but do not include sufficient metadata to describe internal execution flow.

Specifically, the AI service pipeline includes multiple logical stages:

ai_service_fetch_history

ai_service_compact_history

ai_service_submit_message

Currently, telemetry primarily reports total processing duration without granular context at each stage. This black-box characteristic reduces diagnostic visibility and impairs root-cause analysis during performance degradation or failure scenarios.

Because observability limitations directly affect maintainability and operational diagnostics, this issue was categorized as P2 (high priority, non-blocking but strategically important). Enhancing instrumentation would significantly improve traceability and system transparency.

(P2) History Compaction Data Loss

Manual testing identified intermittent loss of contextual information during history compaction. The summarization process, executed by a secondary agent, occasionally omits relevant message content when reducing conversation length.

This defect affects conversational continuity and AI response quality. The suspected root cause is insufficient prompt specificity in the summarizer agent. Refining summarization constraints or including structural guardrails (e.g., key fact preservation requirements) may mitigate information loss.

Because this defect affects functional output quality rather than system availability, it was classified as P2.

(P2) Over-Complicated Custom Message Model

The backend currently includes a custom message model that introduces unnecessary abstraction and causes friction with static type checking. The additional layer provides minimal architectural value and complicates maintainability.

Refactoring to remove or simplify the custom message model would reduce cognitive load and decrease type system conflicts. This issue was categorized as P2 due to its architectural implications and potential to slow future development velocity.

(P3) Inefficient Persona and Project Loading

Personas and project metadata are loaded from file storage during each execution cycle. This introduces unnecessary I/O overhead and increases startup latency.

A more efficient approach would involve caching, dependency injection at initialization, or persistent in-memory storage. Although not critical for MVP functionality, this inefficiency impacts scalability and was therefore classified as P3.

(P3) Excessive Request Turnaround Time (>5 seconds)

Observed request turnaround times frequently exceed five seconds. This latency affects user perception of responsiveness and reduces the conversational fluidity expected in AI-driven systems.

While the system remains functional, latency above five seconds approaches the upper bound of acceptable interactive delay. This issue is actually more of a limitation. Future development will seek to improve system responsiveness.

(P4) LLM Agent Response Time Exceeds Quality Target

The stated quality requirement was consistent response times under five seconds. Current measurements indicate approximately:

~5 seconds for short conversations

~10 seconds for conversations requiring history compaction

This deviation from non-functional performance requirements reflects optimization opportunities rather than a defect that needs to be corrected. It was classified as P4 because the system remains usable, though suboptimal.

Bugs Fixed

The rate limiting mechanism was the only high-priority bug that was addressed. However, it is important to note that numerous higher-severity defects were identified and corrected during active implementation prior to formal documentation. These included issues that would have impacted data integrity, system stability, or core functional workflows and were addressed before the formal Quality Assurance process commenced.

For MVP delivery, performance and reliability were deemed acceptable relative to project scope and presentation requirements.

Known Issues

All other defects listed above remain open at the time of MVP delivery. They are documented within the bug register for future prioritization and remediation.

Most open issues fall into one of three categories:

Minor user interface inconsistencies

Performance optimization opportunities

Backend architectural inefficiencies

None of the unaddressed issues compromise core functionality, security, or data correctness. However, several (particularly observability and compaction accuracy) warrant attention in subsequent development cycles to ensure production readiness.

Bug Triage Process

Bug triage was conducted collaboratively during weekly team meetings. The process involved:

- Identifying reproducible defects
- Assessing business and technical impact
- Assigning severity classification (P-scale)
- Determining remediation timing relative to MVP scope

Team members responsible for specific components provided domain-specific insight into failure modes and technical constraints. One team member contributed prior experience with P-scale incident

classification frameworks, ensuring consistency in severity assignment. Additional input from developers with deeper implementation knowledge ensured accurate assessment of architectural risk.

The triage framework emphasized risk mitigation and MVP viability. Issues affecting system correctness, authentication, or core workflows were prioritized and resolved immediately during development. Remaining issues were evaluated against time constraints and presentation objectives. The team needed to be especially selective in choosing what quality work to perform due to the HR issue that manifested at the beginning of the week.

A formal bug register has now been established to track newly discovered issues. This artifact supports continuous improvement and sprint planning. In retrospect, instituting the bug register at project inception would have improved documentation rigor and reduced reliance on informal tracking mechanisms during early implementation.

Bug identification is inherently iterative. Numerous defects of varying severity were discovered and corrected during the final two weeks of development. The current register represents a stabilized snapshot following major integration efforts rather than the entirety of issues encountered throughout the project lifecycle.

Quality Metrics

Code quality

Code quality was treated as a first-class engineering concern throughout implementation. The team enforced consistent standards across backend and frontend codebases through automated linting, static type checking, and IDE-integrated validation.

Backend Code Quality

Linting & Formatting

The backend uses Ruff for linting and formatting enforcement.

Ruff is:

- Integrated into developer IDEs
- Executed automatically via pre-commit hooks
- Enforced in the CI pipeline

This ensures:

- Consistent formatting
- Elimination of unused imports and variables
- Detection of common Python anti-patterns
- Compliance with PEP8 standards

All backend commits must pass Ruff checks before merging.

Static Type Safety

The backend uses Mypy for static type checking.

Key quality improvements included:

- Strict type checking of service layers
- Full type coverage across Pydantic models
- Elimination of ambiguous model conflicts (notably the earlier “Message” type issue)
- Validation of JWT token structures
- Strong typing of agent orchestration logic

Additionally, enhanced type safety was achieved through structured use of the Pydantic family of models, including:

- Explicit request/response schema validation
- Strict schema enforcement at API boundaries
- Typed model validation before LLM invocation

This approach prevents runtime schema drift and improves maintainability by catching type errors during development rather than at runtime.

Frontend Code Quality

Frontend code quality is enforced through:

- TypeScript static typing
- IDE-integrated linting
- CI-based type checking

Frontend Linting

The frontend uses Biome (successor to Rome) for formatting and linting enforcement within the TanStack-based TypeScript environment.

Biome ensures:

- Consistent formatting
- Elimination of dead code
- Enforcement of TypeScript best practices
- Import consistency
- Prevention of common React anti-patterns

All frontend code must pass Biome validation before integration.

TypeScript Static Typing

TypeScript provides built-in static type enforcement across:

- API request contracts
- TanStack Query hooks
- Conversation schema validation
- Error boundary definitions
- Environment-aware logging logic

During the final testing cycle:

- No TypeScript errors were detected
- All type constraints passed compilation

The combination of Biome + TypeScript provides strong frontend compile-time safety.

Performance benchmarks

Performance evaluation focused on response latency and responsiveness under realistic classroom usage scenarios.

Response Time Measurements

Performance was measured using structured backend logging via Wide-Event tracing.

Each `/generate` request captures:

- ai_service_duration_time_ms
- Total request duration
- Correlation IDs for traceability

This logging allows precise measurement of:

- LLM processing time
- Database interaction time
- End-to-end request duration

Observed Performance Metrics

Measured data showed:

- Average latency: ~8 seconds
- Median latency: ~6.5 seconds
- Fastest response: ~3 seconds
- Slowest response: ~18 seconds

Findings:

- AI inference time accounts for the vast majority of latency.
- Database operations consistently complete in <150ms.
- Latency scales with prompt length and response size.
- Nested summarization calls increase response time for long histories.

The system successfully maintains UI responsiveness during AI calls due to asynchronous FastAPI endpoints and optimistic frontend updates.

However, the strict requirement of:

> 95% of responses ≤ 5 seconds was not met in this MVP implementation.

The primary bottleneck is external LLM inference latency rather than internal architectural inefficiency.

Security considerations

Security was designed across multiple architectural layers.

Authentication Layer

The system uses Neon Auth, which extends Better Auth, for frontend user authentication.

Authentication characteristics:

- Secure session handling
- Token-based authorization
- Integration with Neon-managed identity services

Backend Authorization

Backend endpoints require a JWT Bearer token.

Security enforcement includes:

- Token validation on every protected request
- Extraction of user ID from JWT payload
- Ownership verification before conversation access
- Prevention of cross-user data access

Requests without valid tokens are rejected.

Platform-Level Security

The backend is deployed on Google Cloud Platform (GCP), inheriting:

- Managed TLS encryption
- Secure network isolation
- Cloud IAM enforcement
- Infrastructure-level access controls

TanStack Start Security

TanStack Start is designed as “secure-by-default” and provides:

- Secure API routing patterns
- CSRF protection mechanisms
- XSS mitigation
- Strong input validation
- Type-safe API interactions

By combining type safety with strict API contracts, many injection and validation vulnerabilities are mitigated at compile time.

Accessibility

The front-end application inherits a substantial portion of its accessibility capabilities from the component primitives provided by shadcn/ui, which is built on top of Radix UI. Because these libraries are designed with accessibility as a first-class concern, many compliance-oriented behaviors are embedded at the component level rather than implemented manually.

Radix UI components are constructed using semantic HTML and conform to established WAI-ARIA authoring practices. Interactive elements such as dialogs, dropdown menus, popovers, tabs, and form controls include appropriate ARIA roles, states, and properties by default. This ensures that assistive technologies—such as screen readers—can accurately interpret component purpose, state (e.g., expanded/collapsed), and interaction affordances.

For example, modal dialogs automatically manage aria-labelledby, aria-describedby, and role assignments, ensuring that screen readers correctly announce dialog context. Similarly, form controls are programmatically associated with labels, supporting accessible name computation without additional developer configuration.

A significant accessibility advantage inherited from Radix is robust keyboard interaction support. Components are navigable using standard keyboard controls (e.g., Tab, Shift+Tab, Arrow keys, Enter, Escape), consistent with established accessibility patterns.

Focus management is handled internally by the library, including:

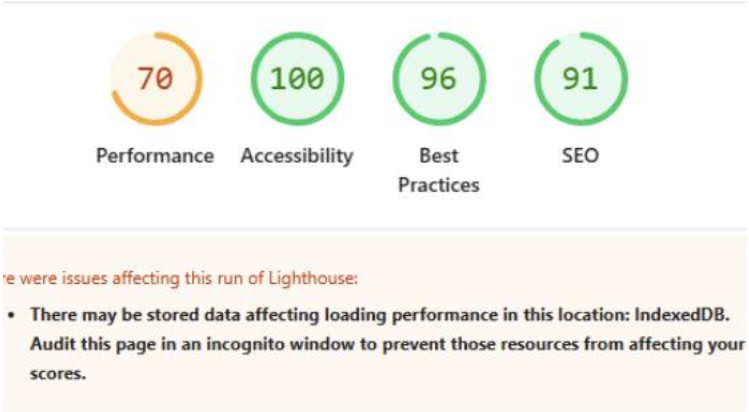
- Logical focus trapping within modal dialogs
- Restoration of focus to the triggering element upon dialog close
- Predictable tab order within composite components

These behaviors are critical for users who rely exclusively on keyboard navigation or alternative input devices.

Shadcn/ui provides accessible default styling patterns, including visible focus indicators for interactive elements. Clear focus outlines improve usability for keyboard users and users with certain visual impairments. Additionally, component styling is compatible with high-contrast modes and can be adapted to meet WCAG contrast ratio guidelines.

Because styling is utility-driven (e.g., Tailwind-based), color contrast and typography can be centrally adjusted to maintain accessibility compliance across the interface.

By leveraging pre-built accessible primitives rather than custom interactive components, the front end reduces the risk of introducing common accessibility defects such as missing ARIA attributes, broken tab order, or inaccessible modal behavior. Accessibility is therefore embedded at the architectural level rather than treated as an afterthought.



System Readiness Assessment

Production readiness

The system is fully functional and already deployed to a web-facing production environment (FGoogle cloud). The Steakholder AI Chat allows students to practice requirements elicitation by conversing with an AI stakeholder agent, and it works end-to-end for the current project and persona. Deployment infrastructure is complete utilizing Dockerfiles and the CI/CD pipeline to handle testing, building, and deployment to Google Cloud Run automatically. A few enhancements are still in progress (streaming responses, AI parameter tuning, and a configuration service for additional personas and projects), but none of these impede the deployment of the application. The core application is functional and ready for production.

Stakeholder readiness

Students should not need any formal training to use the application. The main readiness gap is on the IT/admin side. Whoever sets up the application will need to configure an OpenAI-compatible LLM, supply the necessary credentials as environment variables, and stand up the Neon database and Neon Auth services since the MVP is tied to those providers. A configuration walkthrough covering environment setup and deployment steps should be sufficient, as this is a one-time setup.

Risks and mitigations

The primary risk is vendor lock-in. The MVP depends on Neon for both the database and authentication, and on Google Cloud Run for hosting. An outage from either would directly affect availability. The Dockerized architecture makes migrating to alternative providers feasible, but that work is out of scope for the MVP. A secondary risk is LLM dependency, as the quality of student interactions depends on the model the IT team configures. The OpenAI-compatible interface does allow teams to swap in different model providers without changing application code.

Week 8 readiness

The team is ready for the final presentation. All identified bugs were either addressed or low priority and do not affect core functionality. The demo scenario runs end-to-end without issues. Before presenting, the team should confirm the deployment environment is live, verify LLM credentials are active, and run through the demo at least once in the presentation environment. Having a recorded backup ready would also be a good precaution.